JS-02

Objetivos
Explorar diferentes formas de **armazenamento de dados no navegador**, incluindo:

- **localStorage** – armazenamento persistente no navegador.
- **sessionStorage** – armazenamento temporário durante a sessão.
- **IndexedDB** – base de dados local estruturada.
- **Cache Storage API** – definição de ficheiros disponíveis offline.

Referências externas
https://www.w3schools.com/js/js_api_web_storage.asp
https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API
https://developer.mozilla.org/en-US/docs/Web/API/Cache

localStorage
Permite guardar dados no navegador sem limite de tempo. Os dados permanecem mesmo depois de fechar o navegador.

Exemplo:

```
localStorage.setItem("nome", "Ana");
let nome = localStorage.getItem("nome");
```

---

sessionStorage
É semelhante ao localStorage, mas os dados são apagados quando a sessão termina (fecho da aba ou do navegador).

Exemplo:

```
sessionStorage.setItem("utilizador", "Carlos");
```

IndexedDB
Base de dados local no navegador que permite guardar grandes quantidades de dados estruturados.

Exemplo:

```
let request = indexedDB.open("MinhaBaseDados", 1);
```

Cache Storage API

Permite controlar os recursos que devem ser guardados no navegador para que a aplicação possa funcionar offline.

Exemplo:

```
const CACHE_NAME = "my-app-cache-v1";
const FILES_TO_CACHE = [
  "/",
  "/index.html",
  "/script.js",
  "/offline.html" // Add an offline fallback page
];

self.addEventListener("install", event => {
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(cache => cache.addAll(FILES_TO_CACHE))
  );
});
```

```
self.addEventListener("fetch", event => {
  event.respondWith(
    caches.match(event.request).then(response => {
      return response || fetch(event.request).catch(() => {
        return caches.match("/offline.html");
      });
    })
  );
});

self.addEventListener("activate", event => {
  const cacheWhitelist = [CACHE_NAME];
  event.waitUntil(
    caches.keys().then(cacheNames => {
      return Promise.all(
        cacheNames.map(cacheName => {
          if (!cacheWhitelist.includes(cacheName)) {
            return caches.delete(cacheName);
          }
        })
      );
    })
  );
});
```

Fase 1 - Cria os ficheiros index.html e script.js

```
<!DOCTYPE html>
<html manifest="cache.appcache">

<head>
<meta charset="UTF-8">
<title>Armazenamento no Browser</title>
</head>

<body>

<h1>Teste de Armazenamento</h1>

<input type="text" id="nome" placeholder="Introduz o teu nome">

<button onclick="guardarLocal()">Guardar LocalStorage</button>
<button onclick="guardarSession()">Guardar SessionStorage</button>
<button onclick="lerDados()">Ler Dados</button>

<script src="script.js"></script>

</body>
</html>
```

```
function guardarLocal(){

let nome = document.getElementById("nome").value;

localStorage.setItem("nomeLocal", nome);

console.log("Guardado no localStorage");

}

function guardarSession(){

let nome = document.getElementById("nome").value;

sessionStorage.setItem("nomeSession", nome);

console.log("Guardado no sessionStorage");

}

function lerDados(){

let local = localStorage.getItem("nomeLocal");
let session = sessionStorage.getItem("nomeSession");

console.log("LocalStorage:", local);
console.log("SessionStorage:", session);

}
```

Fase 2 - Adiciona ao ficheiro script.js

```
let db;

let request = indexedDB.open("EscolaDB", 1);

request.onupgradeneeded = function(event){

db = event.target.result;

db.createObjectStore("alunos", { keyPath: "id" });

};

request.onsuccess = function(event){

db = event.target.result;

console.log("Base de dados criada");

};
```

Fase 4 - Guardar dados na IndexedDB

```
function guardarAluno(){

let transaction = db.transaction(["alunos"], "readwrite");

let store = transaction.objectStore("alunos");

let aluno = {

id: 1,
nome: "João",
curso: "Informática"

};

store.add(aluno);

console.log("Aluno guardado na IndexedDB");

}
```

Fase 5 - Criar o service worker para armazenar recursos em cache

Tarefas

1. Guarda um nome utilizando **localStorage** e verifica se permanece após atualizar a página.
2. Guarda um nome utilizando **sessionStorage** e verifica o que acontece ao fechar o navegador.
3. Cria um objeto **aluno** e guarda-o na **IndexedDB**.
4. Consulta os dados em **DevTools → Application → IndexedDB**.
5. Cria o ficheiro **cache.appcache** e adiciona os ficheiros da aplicação.
6. Desliga a internet e verifica se a página continua a abrir.
7. Cria uma interface que permita:
    - adicionar vários alunos na **IndexedDB**
    - mostrar a lista de alunos numa tabela HTML.